Human Verification: `Containers.DirichletClosed`

($\mathbf{Cont}, \otimes, y$) is a Closed Monoidal Category

Machine-generated from `lean/DirichletClosed.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.DirichletClosed` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

The file proves that the Dirichlet tensor $\otimes$ on containers is **closed**: for every container q the functor $- \otimes q$ has a right adjoint, the *internal hom* $[q, r]$. The adjunction is presented as an explicit type equivalence `dirichlet_closure`; all round trips hold by `rfl`. Naturality of currying (in both arguments) and the universal property in counit form (evaluation and uniqueness) are also proved. This is Niu–Spéak Eq. (4.79) in the container presentation.

Contents

1 Overview **2**

2 Auxiliary: type equivalences **2**
2.1 Structure: `TypeEquiv` . . . 2

3 The internal hom **2**
3.1 Definition: `Container.ihom` . . . 2
3.2 Theorem: `Container.ihom_shape_eq` . . . 3
3.3 Theorem: `Container.ihom_pos_eq` . . . 3

4 Currying and uncurrying **3**
4.1 Definition: `dirCurry` . . . 3
4.2 Definition: `dirUncurry` . . . 4

5 Round trips and the main theorem **4**
5.1 Theorem: `dirUncurry_dirCurry` . . . 4
5.2 Theorem: `dirCurry_dirUncurry` . . . 5
5.3 Definition: `dirichlet_closure` . . . 5

6 Naturality **5**
6.1 Definition: `ihomMap` . . . 5
6.2 Theorem: `ihomMap_id` . . . 6
6.3 Theorem: `ihomMap_comp` . . . 6
6.4 Theorem: `dirCurry_naturality_left` . . . 6
6.5 Theorem: `dirCurry_naturality_right` . . . 7
6.6 Theorem: `dirUncurry_naturality_left` . . . 7

7 The counit: evaluation **7**
7.1 Definition: `dirEval` 7
7.2 Theorem: `dirEval_dirCurry` 8
7.3 Theorem: `dirCurry_unique` 8

1 Overview

The central objects are:

- The **internal hom** $[q, r]$: a container whose shapes are the morphisms $q \Rightarrow r$ in **Cont**.
- The **adjunction** $- \otimes q \dashv [q, -]$, given concretely as the currying bijection $(p \otimes q \Rightarrow r) \cong (p \Rightarrow [q, r])$.

The bijection is pure currying: the q-component of a backward position map is packaged *into the shape* of the internal hom (as its own backward position map), and the p-component becomes the curried morphism's backward position map. This split is why the positions of $[q, r]$ are $\Sigma\, t : S_q,\ r[u(t)]$ and not something else.

Every proof is `rfl`: the operations involved are rearrangements of $\Sigma/\times$ telescopes, and Lean 4's definitional η for `Prod`, `Sigma`, and the `ContainerMorphism` structure handles the rest.

2 Auxiliary: type equivalences

2.1 Structure: `TypeEquiv`

Definition 2.1 (Type equivalence)**.** A **type equivalence** $A \simeq B$ is a pair of maps with both round trips:

$$f : A \to B, \quad g : B \to A, \quad g \circ f = \mathrm{id}_A, \quad f \circ g = \mathrm{id}_B.$$

This is a Mathlib-free stand-in for `Equiv`.

Lean.

```
structure TypeEquiv (A : Type u) (B : Type v) where
  toFun    : A → B
  invFun   : B → A
  left_inv : ∀ a, invFun (toFun a) = a
  right_inv: ∀ b, toFun (invFun b) = b
```

Haskell.

```
data TypeEquiv a b = TypeEquiv
  { toFun    :: a → b
  , invFun   :: b → a
  , leftInv  :: forall x. invFun (toFun x) == x    -- propositional
  , rightInv :: forall y. toFun (invFun y) == y
  }
```

3 The internal hom

3.1 Definition: `Container.ihom`

Definition 3.1 (Internal hom for $\otimes$)**.** For containers q and r, the **internal hom** $[q, r]$ is the container:

- *Shapes*: `ContainerMorphism q r` — the hom-set of **Cont**. A shape is a pair (u, ϕ) with $u : S_q \to S_r$ and $\phi_t : r[u(t)] \to q[t]$ for each $t : S_q$.

- *Positions* over shape $f = (u, \phi)$: $\Sigma\, t : S_q,\ r[u(t)]$.

Lean.
```
def Container.ihom (q r : Container) : Container where
  Shape := ContainerMorphism q r
  Pos   := fun f ⇒ (t : q.Shape) * r.Pos (f.onShapes t)
```

Haskell.
```
-- Internal hom [q, r]:
-- Shape  = ContainerMorphism q r
-- Pos f  = Exists t. r.Pos (onShapes f t)   (dependent pair)
data IhomPos q r f = IhomPos { iqShape :: Shape q, irPos :: Pos r (
    onShapes f iqShape) }
-- (Haskell cannot express the dependent type directly.)
```

Fidelity note. The fact that the *shape* type of $[q, r]$ is literally `ContainerMorphism q r` is not a notational convenience — it is the mathematical content (Niu–Spéak Eq. 4.79). The positions over f are pairs (t, i) where $t : S_q$ and $i : r[f(t)]$ is an r-position over the image shape. In Haskell, the dependent type $r.\mathrm{Pos}(f.\texttt{onShapes}(t))$ cannot be expressed directly; the Haskell rendering is therefore approximate.

3.2 Theorem: `Container.ihom_shape_eq`

Theorem 3.2 (Shape of the internal hom)**.** The shape type of $[q, r]$ is definitionally the hom-set:

$$[q, r].\mathrm{Shape} = \mathrm{ContainerMorphism}\, q\, r.$$

Lean.
```
theorem Container.ihom_shape_eq (q r : Container) :
    (Container.ihom q r).Shape = ContainerMorphism q r := rfl
```

3.3 Theorem: `Container.ihom_pos_eq`

Theorem 3.3 (Positions of the internal hom)**.** The position type of $[q, r]$ over a shape $f : q \Rightarrow r$ is:

$$[q, r].\mathrm{Pos}\, f = \Sigma\, t : S_q,\ r[f(t)].$$

Lean.
```
theorem Container.ihom_pos_eq (q r : Container)
    (f : ContainerMorphism q r) :
    (Container.ihom q r).Pos f
      = ((t : q.Shape) * r.Pos (f.onShapes t)) := rfl
```

4 Currying and uncurrying

4.1 Definition: `dirCurry`

Definition 4.1 (Currying)**.** Given $F : p \otimes q \Rightarrow r$, define $\texttt{dirCurry}(F) : p \Rightarrow [q, r]$ by:

- *On shapes*: $s \mapsto \{$shape map: $t \mapsto F(s, t)$; backward position map: $t, i \mapsto (\texttt{F.onPos}(s, t, i)).2\}$. (The q-component of F's backward position map becomes the shape's own backward position map.)
- *On positions* (backward): $(s, (t, i)) \mapsto (\texttt{F.onPos}(s, t, i)).1$. (The p-component of F's backward position map.)

Lean.

```
def dirCurry {p q r : Container}
    (F : ContainerMorphism (p |*| q) r) :
    ContainerMorphism p (Container.ihom q r) where
  onShapes := fun s ⇒
    { onShapes := fun t   ⇒ F.onShapes (s, t)
      onPos    := fun t i ⇒ (F.onPos (s, t) i).2 }
  onPos := fun s x ⇒ (F.onPos (s, x.1) x.2).1
```

Haskell.

```
dirCurry :: ContainerMorphism (p `Dir` q) r
         → ContainerMorphism p (Ihom q r)
dirCurry f = Morph
  { onShapes = \s → Morph
      { onShapes = \t   → onShapes f (s, t)
      , onPos    = \t i → snd (onPos f (s, t) i) }
  , onPos    = \s (t, i) → fst (onPos f (s, t) i) }
```

4.2 Definition: `dirUncurry`

Definition 4.2 (Uncurrying)**.** Given $G : p \Rightarrow [q, r]$, define $\texttt{dirUncurry}(G) : p \otimes q \Rightarrow r$ by:

- *On shapes*: $(s, t) \mapsto (G(s)).(t)$, i.e. apply the morphism $G(s) : q \Rightarrow r$ to the shape t.
- *On positions* (backward): $(s, t), i \mapsto (G.\texttt{onPos}(s, (t, i)),\ (G(s)).\texttt{onPos}(t, i))$, producing a pair of a p-position and a q-position.

Lean.

```
def dirUncurry {p q r : Container}
    (G : ContainerMorphism p (Container.ihom q r)) :
    ContainerMorphism (p |*| q) r where
  onShapes := fun st ⇒ (G.onShapes st.1).onShapes st.2
  onPos    := fun st i ⇒
    (G.onPos st.1 ⟨st.2, i⟩, (G.onShapes st.1).onPos st.2 i)
```

Haskell.

```
dirUncurry :: ContainerMorphism p (Ihom q r)
           → ContainerMorphism (p `Dir` q) r
dirUncurry g = Morph
  { onShapes = \(s, t) → onShapes (onShapes g s) t
  , onPos    = \(s, t) i →
      (onPos g s (t, i), onPos (onShapes g s) t i) }
```

5 Round trips and the main theorem

5.1 Theorem: `dirUncurry_dirCurry`

Theorem 5.1 (Uncurrying undoes currying)**.** For all $F : p \otimes q \Rightarrow r$:

$$\texttt{dirUncurry}(\texttt{dirCurry}(F)) = F.$$

Holds by `rfl`.

Lean.

```
theorem dirUncurry_dirCurry {p q r : Container}
    (F : ContainerMorphism (p |*| q) r) :
    dirUncurry (dirCurry F) = F := rfl
```

Haskell.
```haskell
-- dirUncurry (dirCurry f) = f    -- by eta and product eta
```

5.2 Theorem: `dirCurry_dirUncurry`

Theorem 5.2 (Currying undoes uncurrying)**.** For all $G : p \Rightarrow [q, r]$:

$$\texttt{dirCurry}(\texttt{dirUncurry}(G)) = G.$$

Holds by `rfl`.

Lean.
```
theorem dirCurry_dirUncurry {p q r : Container}
    (G : ContainerMorphism p (Container.ihom q r)) :
    dirCurry (dirUncurry G) = G := rfl
```

Haskell.
```haskell
-- dirCurry (dirUncurry g) = g    -- by eta for ContainerMorphism
   structure
```

5.3 Definition: `dirichlet_closure`

Definition 5.3 (The Dirichlet tensor is closed)**.** For all containers p, q, r, there is a type equivalence:

$$(p \otimes q \Rightarrow r) \;\simeq\; (p \Rightarrow [q, r]).$$

This is the adjunction $- \otimes q \dashv [q, -]$, or equivalently, $(\mathbf{Cont}, \otimes, y)$ is a **closed monoidal category**.

This is Niu–Spéak, *Polynomial Functors*, Eq. (4.79), in the container presentation.

Declared as a `def` (not a `theorem`) because a type equivalence is data (two maps plus two round-trip proofs).

Lean.
```
def dirichlet_closure (p q r : Container) :
    ContainerMorphism (p |*| q) r ≃> ContainerMorphism p (Container.
        ihom q r) where
  toFun    := dirCurry
  invFun   := dirUncurry
  left_inv := dirUncurry_dirCurry
  right_inv:= dirCurry_dirUncurry
```

Haskell.
```
dirichletClosure :: TypeEquiv
  (ContainerMorphism (p `Dir` q) r)
  (ContainerMorphism p (Ihom q r))
dirichletClosure = TypeEquiv
  { toFun    = dirCurry
  , invFun   = dirUncurry
  , leftInv  = dirUncurryCurry
  , rightInv = dirCurryUncurry }
```

6 Naturality

6.1 Definition: `ihomMap`

Definition 6.1 (Functorial action of the internal hom)**.** For $\psi : r \Rightarrow r'$, the action $\texttt{ihomMap}(\psi) : [q, r] \Rightarrow [q, r']$:

- On shapes: postcompose, $f \mapsto f \cdot \psi$. (A shape of $[q, r]$ is a morphism $q \Rightarrow r$; postcompose with ψ to get a morphism $q \Rightarrow r'$.)
- On positions (backward): given a position (t, i) of $[q, r']$ over $f \cdot \psi$ (where $i : r'[\psi(f(t))]$), push i back through $\psi^\sharp$ to get an r-position $\psi^\sharp_{f(t)}(i)$, giving $(t,\, \psi^\sharp_{f(t)}(i))$ of $[q, r]$.

Lean.

```
def ihomMap {q r r' : Container} (psi : ContainerMorphism r r') :
    ContainerMorphism (Container.ihom q r) (Container.ihom q r') where
  onShapes := fun f ⇒ f.comp psi
  onPos    := fun f x ⇒ ⟨x.1, psi.onPos (f.onShapes x.1) x.2⟩
```

Haskell.

```
ihomMap :: ContainerMorphism r r'
        → ContainerMorphism (Ihom q r) (Ihom q r')
ihomMap psi = Morph
  { onShapes = \f → f `compM` psi
  , onPos    = \f (t, i) → (t, onPos psi (onShapes f t) i) }
```

6.2 Theorem: `ihomMap_id`

Theorem 6.2 (Internal hom preserves identities)**.** $\texttt{ihomMap}(\mathrm{id}_r) = \mathrm{id}_{[q,r]}$. Holds by `rfl`.

Lean.

```
theorem ihomMap_id (q r : Container) :
    ihomMap (q := q) (ContainerMorphism.id r)
      = ContainerMorphism.id (Container.ihom q r) := rfl
```

6.3 Theorem: `ihomMap_comp`

Theorem 6.3 (Internal hom preserves composition)**.** $\texttt{ihomMap}(\psi \cdot \psi') = \texttt{ihomMap}(\psi) \cdot \texttt{ihomMap}(\psi')$. Holds by `rfl`.

Lean.

```
theorem ihomMap_comp {q r r' r'' : Container}
    (psi : ContainerMorphism r r') (psi' : ContainerMorphism r' r'') :
    ihomMap (q := q) (psi.comp psi')
      = (ihomMap psi).comp (ihomMap psi') := rfl
```

6.4 Theorem: `dirCurry_naturality_left`

Theorem 6.4 (Naturality in p (currying direction))**.** For $h : p' \Rightarrow p$ and $F : p \otimes q \Rightarrow r$:

$$\texttt{dirCurry}((h \otimes \mathrm{id}_q) \cdot F) \;=\; h \cdot \texttt{dirCurry}(F).$$

Holds by `rfl`.

Lean.

```
theorem dirCurry_naturality_left {p p' q r : Container}
    (h : ContainerMorphism p' p)
    (F : ContainerMorphism (p |*| q) r) :
    dirCurry ((Container.dir2 h (ContainerMorphism.id q)).comp F)
      = h.comp (dirCurry F) := rfl
```

Haskell.

```
-- naturality_left: dirCurry ((h `tensor` id) `compM` f) = h `compM`
   dirCurry f
```

6.5 Theorem: `dirCurry_naturality_right`

Theorem 6.5 (Naturality in r (currying direction))**.** For $F : p \otimes q \Rightarrow r$ and $\psi : r \Rightarrow r'$:

$$\texttt{dirCurry}(F \cdot \psi) \;=\; \texttt{dirCurry}(F) \cdot [q, \psi].$$

Holds by `rfl`.

Lean.

```
theorem dirCurry_naturality_right {p q r r' : Container}
    (F : ContainerMorphism (p |*| q) r)
    (psi : ContainerMorphism r r') :
    dirCurry (F.comp psi) = (dirCurry F).comp (ihomMap psi) := rfl
```

Haskell.

```
-- naturality_right: dirCurry (f `compM` psi) = dirCurry f `compM`
   ihomMap psi
```

6.6 Theorem: `dirUncurry_naturality_left`

Theorem 6.6 (Naturality in p (uncurrying direction))**.** For $h : p' \Rightarrow p$ and $G : p \Rightarrow [q, r]$:

$$\texttt{dirUncurry}(h \cdot G) \;=\; (h \otimes \mathrm{id}_q) \cdot \texttt{dirUncurry}(G).$$

Holds by `rfl`.

Lean.

```
theorem dirUncurry_naturality_left {p p' q r : Container}
    (h : ContainerMorphism p' p)
    (G : ContainerMorphism p (Container.ihom q r)) :
    dirUncurry (h.comp G)
      = (Container.dir2 h (ContainerMorphism.id q)).comp
          (dirUncurry G) := rfl
```

7 The counit: evaluation

7.1 Definition: `dirEval`

Definition 7.1 (Evaluation morphism)**.** The **counit** of the adjunction $- \otimes q \dashv [q, -]$ is the **evaluation** morphism

$$\mathrm{ev} : [q, r] \otimes q \Rightarrow r,$$

obtained by uncurrying the identity on $[q, r]$. Explicitly:

- On shapes: $(f, t) \mapsto f(t)$, i.e. *apply* the morphism f stored in the first component to the shape t.
- On positions (backward): $i \mapsto ((t, i),\ f_t^\sharp(i))$, splitting an r-position i into a $[q, r]$-position and a q-position.

Lean.

```
def dirEval (q r : Container) :
    ContainerMorphism (Container.ihom q r |*| q) r :=
  dirUncurry (ContainerMorphism.id (Container.ihom q r))
```

Haskell.

```
dirEval :: ContainerMorphism (Ihom q r `Dir` q) r
dirEval = dirUncurry idMorph
-- Concretely:
--   onShapes (f, t) = onShapes f t
--   onPos (f, t) i  = ((t, i), onPos f t i)
```

7.2 Theorem: `dirEval_dirCurry`

Theorem 7.2 (Universal property: existence)**.** For every $F : p \otimes q \Rightarrow r$:

$$(\texttt{dirCurry}(F) \otimes \mathrm{id}_q) \cdot \mathrm{ev} \;=\; F.$$

Holds by `rfl`. This is the existence half of the universal property of the counit.

Lean.

```
theorem dirEval_dirCurry {p q r : Container}
    (F : ContainerMorphism (p |*| q) r) :
    (Container.dir2 (dirCurry F) (ContainerMorphism.id q)).comp
        (dirEval q r) = F := rfl
```

Haskell.

```
-- (dirCurry f `tensor` id) `compM` dirEval = f
```

7.3 Theorem: `dirCurry_unique`

Theorem 7.3 (Universal property: uniqueness)**.** If $G : p \Rightarrow [q, r]$ satisfies $(G \otimes \mathrm{id}_q) \cdot \mathrm{ev} = F$, then $G = \texttt{dirCurry}(F)$.

Lean.

```
theorem dirCurry_unique {p q r : Container}
    (F : ContainerMorphism (p |*| q) r)
    (G : ContainerMorphism p (Container.ihom q r))
    (h : (Container.dir2 G (ContainerMorphism.id q)).comp
           (dirEval q r) = F) :
    G = dirCurry F
```

Haskell.

```
-- uniqueness: if (g `tensor` id) `compM` ev = f, then g = dirCurry f.
```

Fidelity note. The uniqueness proof is the only non-`rfl` proof in the file: it uses `subst` to substitute the hypothesis h, reducing to $\texttt{dirCurry}(\texttt{dirUncurry}(G)) = G$, which is exactly `dirCurry_dirUncurry` read backwards. So the mathematical content is still the round-trip, applied once.